

Server-Factory-Core-Framework

Der gemeinsame Motor hinter jedem Server Factory.

Zusammenfassung

Core Framework ist das Kotlin-Grundgerüst, das die Server Factory-Familie von Bereitstellungstools trägt. Es stellt den gemeinsamen Motor und die Abstraktionen bereit, auf denen Projekte wie Mail Server Factory aufbauen, sodass jede „Fabrik“ auf einer erprobten Grundlage aufsetzt, statt Bereitstellungsprimitive neu zu implementieren.

Kurzbeschreibung

Das gemeinsame Kotlin-Framework im Fundament des Server Factory-Ökosystems. Es liefert den einheitlichen Bereitstellungsmotor, Verbindungsabstraktionen und die Installationsschritt-Logik, die von nachgelagerten Fabriken (Mail Server Factory, Web Service Factory, SonarQube Factory u. a.) genutzt werden.

Ausführliche Beschreibung

Core Framework ist das unscheinbare, aber entscheidende Ingenieurswerk, das die gesamte Server-Factory-Familie erst möglich macht: der wiederverwendbare Motor, auf dem jedes einzelne „Factory“-Produkt (Mail Server Factory, Web Service Factory, SonarQube Factory, Caching Proxy Factory) aufbaut. Der Server Factory-Ansatz ist deklarativ – der Nutzer beschreibt die gewünschte Infrastruktur als Konfiguration, und eine Factory interpretiert diese Beschreibung, um Software auf einem Zielsystem zu installieren und zu initialisieren. Core Framework ist der Ort, an dem die für dieses Muster gemeinsamen Mechanismen tatsächlich leben: die Verbindungs- und Transportabstraktionen, die jede Art von Zielsystem erreichen, das Installationsschritt-Modell, das *wie* Software bereitgestellt

wird, kodiert, sowie die gemeinsame Infrastruktur, die jede Factory sonst selbst entwickeln müsste. Es ist die Antwort auf eine strukturelle Frage, der sich jede Multi-Produkt-Toolchain irgendwann stellen muss: *Wo gehört der gemeinsame Motor hin?* – und diese Antwort einmal richtig zu geben, hält die Familie zusammen, statt sie in vier leicht unterschiedliche Bereitsteller zerfallen zu lassen. Durch die Zentralisierung in einem einzigen Kotlin-Framework vermeidet die Familie, Bereitstellungslogik über die Produkte hinweg zu duplizieren, und sorgt für konsistentes Verhalten: Eine in Core Framework verbesserte Verbindungstechnologie oder ein optimiertes Installationsprimitiv kommt jeder nachgelagerten Factory zugute. Es besteht fast ausschließlich aus Kotlin (ca. 990 KB Kotlin mit einer dünnen Shell-Schicht), was seiner Rolle als Code-Bibliothek und nicht als Sammlung von Skripten entspricht. Nachgelagerte Repositories verweisen darauf als ihre kanonische Abhängigkeit (Parallels-Utills, Qemu-Utills, Utills und die Definitions-Pakete referenzieren das Core-Framework-Repository als Drehscheibe des Ökosystems). Die README-Datei ist bewusst knapp gehalten – es handelt sich um Infrastruktur für andere Projekte, versioniert über `version.txt/version_code.txt` – und geht der späteren AI-Arbeit voraus, was es zu einem Teil des etablierten DevOps-Toolchain-Erbes der Organisation macht.

Warum wir es entwickelt haben

Jedes Bereitstellungstool benötigt denselben Kern: Methoden, um Ziele anzubinden, und Schritte, um Software zu installieren und zu konfigurieren. Würde man dies für jedes Produkt neu entwickeln, entstünden fragmentierte Verhaltensweisen und vervielfachte Fehlerquellen. Core Framework zentralisiert diese Logik, sodass alle Fabriken auf einen zuverlässigen Motor zurückgreifen.

Warum es ein Game-Changer ist

Es ist der Punkt mit der größten Hebelwirkung in der gesamten Familie: Eine hier gehärtete Verbindungstechnologie oder ein verbessertes Installationsprimitiv verbreitet diese Korrektheit und Funktionalität sofort auf alle Fabriken – die gesamte Toolchain profitiert von einer einzigen Investition. Es verkörpert die Philosophie *„Einmal entwickeln, überall nutzen“* dort, wo sie den größten Nutzen bringt: in der Basisschicht der Infrastrukturautomatisierung, wo eine Korrektur am richtigen Ort alles Nachgelagerte mitbehebt.

Was innovativ ist

- Ein einziges wiederverwendbares Bereitstellungs-Framework, das Logik für Verbindungen und Installationsschritte abstrahiert.
- Klare Trennung zwischen der Engine (Core Framework) und produktspezifischen Fabriken.
- Versionsgebundene Distribution (`version.txt/version_code.txt`) für reproduzierbare Nutzung.

Herausforderungen & Lösungen

- **Vermeidung doppelter Bereitstellungslogik:** gelöst durch die Auslagerung gemeinsamer Mechanismen in ein Framework, das von allen Fabriken genutzt wird.
- **Konsistentes Verhalten über alle Produkte hinweg:** gelöst durch gemeinsame Abstraktionen, sodass Verbindungstypen und Schritte überall identisch funktionieren.
- **(UNBESTÄTIGT):** Bestimmte interne APIs sind in der öffentlichen README nicht dokumentiert; Schnittstellendetails gelten über das „von den Fabriken genutzte gemeinsame Framework“ hinaus als unbestätigt.

Technologie-Stack (Begründung & Umsetzung)

- **Kotlin** — das gesamte Framework (~990 KB); die Sprache der Server Factory-Familie.
- **Shell** — minimale unterstützende Skripte.
- **Gradle** — Build-Toolchain (konsistent mit der familienüblichen Nutzung von `./gradlew`).

Hinweis: GitHub kennzeichnet das Repository als Fork innerhalb der Server-Factory-Organisation. Nicht AI-zentriert; dient als Rückgrat der Bereitstellungs-Toolchain.